ספר פרויקט

משחק דמקה ב־Java וב־JavaFX

תיעוד תכנון, מימוש והפעלה

משחק רשת TCP, חשבונות ופרופיל משתמש

שמות הסטודנטים: _____

שם המנחה: _____

מוסד הלימודים: _____

אוקטובר 2026

תקציר ותוכן עניינים

הפרויקט הוא משחק דמקה ב־Java וב־JavaFX. הוא כולל משחק מול BOT על המחשב המקומי ומשחק בין שני לקוחות המחוברים לשרת TCP. השרת מנהל את החדרים ואת מצב משחק הרשת, מאמת מהלכים ושומר חשבונות ותוצאות ב־SQLite. לכל שחקן מחובר יש פרופיל ובו סטטיסטיקה והיסטוריית משחקים.

הספר שומר על מבנה ספר הפרויקט של הגרסה המקומית ומעדכן אותו למערכת הנוכחית. לצד פרקי המשחק, האלגוריתם והממשק מופיעים פירוט התקשורת, מנגנון החשבונות ומסד הנתונים. פרק השימוש ב־AI והתרומה האישית מיועד להשלמה עם המגישים.

1. מבוא ומטרות 3
2. כללי המשחק ודרישות 4
3. מבנה המערכת 5
4. מודל הנתונים והמחלקות 6
5. מהלך, תור ואכילה 7
6. השחקן הממוחשב 8
7. ממשק המשתמש 9
8. בדיקות והערכה 10
9. הפעלה וסביבת פיתוח 11
10. סיכום ומקורות 13
11. תרשים UML וקשרים בין המחלקות 14
12. פירוט מעמיק של המחלקות 16
13. זרימת פעולה במערכת 18
14. אלגוריתם Minimax וגיזום Alpha-Beta 20
15. רמות קושי ופונקציית הערכה 21
16. מסכי המערכת 22
17. מצבי משחק חזותיים 26
18. אתגרים ותהליך העבודה 27
19. תקשורת TCP וניהול חיבורים 28
20. חשבונות, SQLite ופרופיל משתמש 32
21. שימוש בכלי AI והתרומה האישית 35

1. מבוא ומטרות

1.1 רקע

דמקה היא משחק לשני שחקנים שבו ההחלטה על מהלך תלויה במיקום הכלים, באפשרויות האכילה ובתגובה הצפויה של היריב. הפרויקט מתרגם את הכללים למודל תוכנה ולממשק אינטראקטיבי. בתחילת הדרך שני בני אדם שיחקו באותו מחשב. בגרסה הנוכחית כל אדם מפעיל לקוח JavaFX משלו, והשרת מאפשר להם לשחק יחד דרך הרשת.

1.2 מטרות הפרויקט

- להציג לוח ברור ולאפשר בחירת כלי ומשבצת יעד באמצעות העכבר, עם משוב ואנימציה.
- לרכז את חוקי התנועה, האכילה, רצף האכילות, ההכתרה והניצחון בתוך Game.
- לאפשר יצירת חדר וקבלת קוד הזמנה, הצטרפות של שחקן נוסף וסנכרון שני הלקוחות.
- להוסיף הרשמה והתחברות, חשבון מתמשך, סטטיסטיקה והיסטוריה הנשמרות בשרת.
- לשמור משחק מקומי מול BOT בשלוש רמות קושי, גם בלי שרת זמין.

1.3 היקף

משחק הרשת מיועד בשלב זה למחשבים באותה רשת מקומית. כתובת השרת והפורט ניתנים להגדרה בלי להציג שדות טכניים במסך המשחק. אין בפרויקט פריסה בענן, הצפנת TLS, שעון משחק או תיקו אוטומטי. משתמש שאינו מחובר לחשבון יכול לתרגל מול BOT, אך התוצאה אינה נשמרת.

1.4 תוצר

התוצר כולל לקוחות גרפיים, שרת Java עצמאי, מסד SQLite ובדיקות אוטומטיות. הבנייה נעשית באמצעות Gradle. המחלקה Main מפעילה את DamkaFX, ו־DamkaServer היא נקודת הכניסה של השרת. הספר מתאר את הקוד הנוכחי ואת גבולות המערכת, ולא רק את המטרות המתוכננות.

2. כללי המשחק ודרישות

2.1 מצב פתיחה

הלוח מיוצג כמערך בגודל 8×8. לכל שחקן 12 כלים על המשבצות הכהות בשלוש השורות הקרובות אליו. שחקן X מתחיל בתחתית ונע לכיוון השורה העליונה; שחקן O מתחיל למעלה ונע מטה. X פותח. אינדקסי השורות והעמודות הם 0 עד 7, ולכן כלי X במיקום (5,0) יכול בפתיחה לנוע אל (4,1).

2.2 תנועה ואכילה

- כלי רגיל נע משבצת אחת באלכסון קדימה אל משבצת פנויה.
- אכילה נעשית בקפיצה של שתי משבצות מעל כלי יריב אל משבצת פנויה. כלי רגיל אוכל קדימה בלבד.
- כאשר קיימת אכילה, מהלך רגיל אינו מותר. אחרי אכילה שיש אחריה אכילה נוספת, חובה להמשיך עם אותו כלי.
- מלך נע ואוכל בשני הכיוונים. המלך במימוש זה אינו מעופף: צעד רגיל הוא משבצת אחת וקפיצת אכילה היא שתי משבצות.

2.3 הכתרה וסיום

כלי x שמגיע לשורה 0 וכלי o שמגיע לשורה 7 מוכתרים למלכים. ההכתרה מתבצעת בתוך Game לפני בדיקת המשך האכילה; לכן כלי שהוכתר במהלך אכילה עשוי להידרש להמשיך לאחור כמלך. ניצחון נקבע כאשר ליריב אין מהלך חוקי. במשחק רשת גם פרישה, עזיבה או ניתוק במהלך משחק פעיל מעניקים ניצחון ליריב.

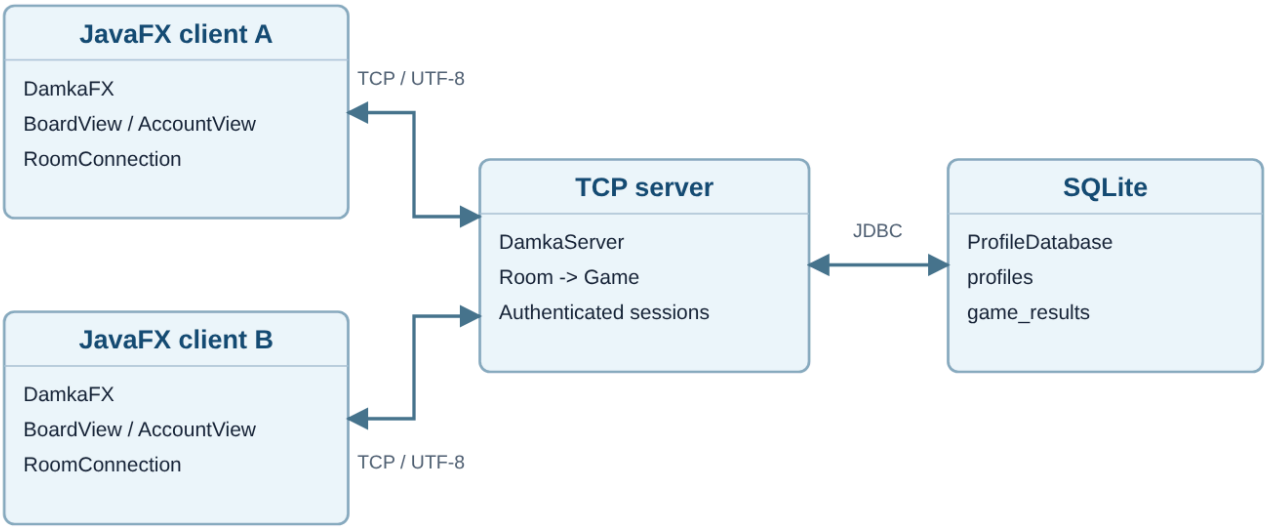
2.4 דרישות שימוש

משחק רשת דורש שני חשבונות שונים, חדר פעיל ושני חיבורים לשרת. הלקוח מציג את השיוך X/O ואת התור שנשלח מהשרת. BOT יכול לפעול ללא חשבון; שמירת תוצאת BOT דורשת שהמשחק יתחיל ויסתיים באותו חשבון ובאותו חיבור מאומת.

3. מבנה המערכת

3.1 חלוקת אחריות

המערכת מורכבת משכבת תצוגה, שכבת תקשורת, לוגיקת משחק ושכבת שמירה. הלקוח מקבל קלט ומציג מצב. השרת מזהה בקשות ומנהל את החדרים. Game קובעת את חוקיות המהלך ואת התוצאה, ו־ProfileDatabase שומרת חשבונות ותוצאות. הפרדה זו מאפשרת לבדוק את הכללים ואת פרוטוקול הרשת גם ללא חלון גרפי.



איור 1 — שני לקוחות, שרת TCP משותף ומסד נתונים בצד השרת.

3.2 שני מקומות לניהול משחק

במשחק מול אדם אחר, אובייקט Game נמצא בתוך Room בשרת. כל לקוח מחזיק תמונת לוח לציור בלבד. במשחק מול BOT, DamkaFX מחזיקה botGame מקומי, ו־BotPlayer בוחרת מהלך על עותקי המשחק. בשני המצבים התצוגה והאנימציה ניתנות על ידי BoardView.

3.3 בחירות תכנון

פרוטוקול התקשורת משתמש בשורות טקסט, ומצב הלוח מקודד ב־64 סימנים. SQLite נבחרה כדי לשמור מידע בקובץ בלי להפעיל שרת מסד נוסף. חוקי המשחק מרוכזים ב־Game; בדיקות הממשק עוסקות בזמינות פעולה ובבחירה, ובדיקות Room עוסקות בחדר, בזהות ובמבנה הבקשה.

4. מודל הנתונים והמחלקות

4.1 Board ו־Piece

Board עוטפת מערך דו־ממדי של Piece. היא מאתחלת את כלי הפתיחה, מספקת גישה לפי שורה ועמודה, בודקת גבולות ומשבצת פנויה ומבצעת הזזה או הסרה שהתבקשו ממנה. היא אינה מחליטה אם מהלך חוקי ואינה אחראית להכתרה. Piece מחזיקה סימן יחיד: x/o הם כלים רגילים, X/O הם מלכים. המתודות belongsToPlayer ו־isOpponent בודקות שייכות בלי תלות בגודל האות.

4.2 Position, Player ו־Game.Move

Position שומרת זוג קואורדינטות. Player שומרת סימן, שם ודגל המציין BOT. אלה ישויות של מודל המשחק; Player אינה חשבון במסד. Game.Move מתארת מהלך חוקי: מקור, יעד, מיקום כלי שנאכל אם יש, וסימן הכלי בזמן תחילת התנועה. נתונים אלו משמשים גם לאנימציה.

4.3 Game ו־Room

Game מחזיקה לוח, שני שחקנים, שחקן נוכחי, מיקום המחויב להמשך אכילה, מהלך אחרון ושגיאת המהלך האחרון. Room מחזיקה Game של משחק מקוון, קוד חדר, שני Socket, ערוצי כתיבה ושמות חשבונות. Room מחברת בין מודל המשחק לבין המשתתפים, ואינה משכפלת את כללי התנועה.

4.4 חשבון ופרופיל

החשבון נשמר בטבלת profiles. ProfileData היא רשומת נתונים המתארת את הפרופיל וכוללת שם משתמש, שתי קבוצות Stats ורשימת HistoryEntry. היא נשלחת ללקוח ומוצגת ב־ProfileWindow. ProfileDatabase אחראית לפעולות SQL, ו־AccountView מציגה את טופס ההזדהות. פירוט השדות והקשרים נמצא בפרק 20.

5. מהלך, תור ואכילה

5.1 אימות מהלך במקום אחד

Game.move מקבלת מקור ויעד. תחילה היא בודקת קלט חסר ומוודאת שלא נבחר כלי אחר באמצע רצף אכילות. לאחר מכן היא מחפשת את זוג המקור והיעד בתוך getValidMoves של השחקן הנוכחי. אם לא נמצא מהלך כזה, היא מחזירה false ושומרת הודעת שגיאה. הלוח והתור אינם משתנים.

5.2 אכילה מחייבת

getValidMoves סורקת את כלי השחקן ואוספת קפיצות אכילה וצעדים רגילים. אם נמצאה אכילה אחת לפחות, היא מחזירה רק אכילות. getPieceMoves מרכזת את בדיקות הכיוון, המרחק, גבולות הלוח, היעד הפנוי והיריב באמצע הקפיצה. אותו מקור למהלכים משמש את המהלך האנושי, בדיקות הניצחון וה־BOT.

5.3 ביצוע ורצף אכילות

כאשר המהלך התקבל, Game מסירה את הכלי שנאכל, מזיזה את הכלי ומכתירה אותו אם הגיע לקצה. אחר כך היא שומרת lastMove. אם הייתה אכילה ואפשר לאכול שוב, היא שומרת את מיקום היעד ב־captureContinuationPosition ומשאירה את התור. אחרת היא מנקה את המיקום ומחליפה שחקן. רשימת המהלכים הבאה מוגבלת לכלי שחייב להמשיך.

5.4 ניצחון ומשחק רשת

getWinnerSymbol מחזירה X או O כאשר הצד האחר אינו יכול לנוע, ו־'\0' כאשר אין מנצח. במשחק רשת Room מבצעת לפני Game בדיקות של החיבור המבקש, מספר השחקנים, התור ומבנה הקואורדינטות. לאחר אישור היא משדרת לשני המשתתפים את המהלך, הלוח והתור; לאחר דחייה היא שולחת ERROR למבקש בלבד.

6. השחקן הממוחשב

6.1 בחירת מהלך

BotPlayer.findBestMove מקבלת Game, עומק חיפוש וסימן BOT. היא מבקשת את המהלכים החוקיים מ־Game. עבור כל אפשרות היא יוצרת עותק של המשחק, מבצעת בו את המהלך ומעריכה את התוצאה בעזרת Minimax. היא מחזירה את המהלך בעל הציון הגבוה ביותר מנקודת המבט של ה־BOT. בשוויון נשמרת האפשרות הראשונה שנמצאה.

6.2 חיפוש על עותקים

בנאי ההעתקה של Game יוצר Board חדשה ובה Piece חדשות, משכפל את השחקנים, התור ומיקום המשך האכילה. לכן בדיקת מהלך עתידי אינה מזיזה כלי בלוח שהמשתמש רואה. גם רצף אכילות נשמר בעותק, כדי שהחיפוש לא יאפשר החלפת כלי בניגוד לכללים.

6.3 שילוב במשחק המקומי

במצב BOT האדם משחק X וה־BOT משחק O. לאחר האנימציה של מהלך האדם, הממשק בודק תוצאה ותור. אם התור שייך ל־BOT, הוא מבקש מהלך ומפעיל את אותו מסלול applyLocalMove שמשתמש ב־Game.move. אם נשארה אכילה עם אותו כלי, ה־BOT ממשיך עד השלמת תורו.

6.4 קושי ומגבלות

שלוש רמות הקושי משתמשות באותו Minimax ובאותו גיזום Alpha-Beta. ההבדל הוא עומק 1, 2 או 3. חיפוש BOT הנוכחי נקרא מחוט JavaFX, ולכן חיפוש גדול יותר עלול להשפיע על תגובת הממשק. פעולות הרשת, לעומת זאת, מתבצעות ברקע. פירוט האלגוריתם וההערכה מופיע בפרקים 14 עד 15.

7. ממשק המשתמש

7.1 כניסה וניווט

מסך הפתיחה מציג Log in, Create account ו־Play BOT offline. לאחר הצלחת ההזדהות עוברים למסך המשחק. מסך זה כולל כותרת, תור ומצב חיבור, לוח, כרטיס משחק עם חבר וכרטיס תרגול מול BOT. כפתור Profile פותח חלון נפרד; סגירתו מחזירה את המיקוד לחלון המשחק שנותר פתוח.

7.2 אינטראקציה

לחיצה על כלי בוחרת אותו ולחיצה על משבצת יעד פנויה מבקשת מהלך. במצב BOT הבחירה נעזרת ב־Game.canSelectPiece במצב מקוון הלקוח בודק בעלות לצורך בחירה ואת זמינות הפעולה, אבל השרת קובע אם המהלך מותר. כאשר בקשה או אנימציה עדיין מתבצעות, הממשק מונע שליחת מהלך נוסף.

7.3 עיצוב ומשוב

BoardView מציירת לוח בגוני קרם וירוק יער. לכלי X גוון ירוק ולכלי O גוון אלמוג. בחירה מסומנת בטבעת זהובה, ומלך מסומן במסגרת זהובה עבה. CSS נמצא ב־damka.css. שגיאה מופיעה בכרטיס צף שניתן לסגור; היא אינה מוסיפה שורה לפריסה ואינה מקטינה את הלוח. פעולה מוצלחת מנקה שגיאה קודמת.

7.4 אנימציה ושינוי גודל

אותו BoardView משמש את BOT ואת משחק הרשת. Timeline מנפישה את הכלי במשך כ־320 מילישניות; אכילה מוסיפה FadeTransition לכלי שנאכל. נקודות התנועה מחושבות ביחס לתאי הלוח ולשכבת האנימציה, ולכן שינוי גודל או גלילה אינם משנים את מיקום המקור והיעד. בחלון צר כרטיסי הצד עוברים מתחת ללוח.

8. בדיקות והערכה

8.1 בדיקות אוטומטיות

הבדיקות משתמשות ב־JUnit 5. הן בוחנות את Game, תקשורת TCP אמיתית ב־loopback ואת SQLite בקבצים זמניים. בדיקות התקשורת כוללות שני לקוחות, תורות, בקשות שגויות והמשך פעולה לאחר שגיאה. בדיקות החשבונות כוללות הרשמה, סיסמה שגויה, זהות החשבון, תוצאות ופרופיל.

מחלקת בדיקות	כמות	תוצאה
AuthProfileTest	14	עברו
GameRulesTest	11	עברו
ProfileDatabaseTest	7	עברו
RoomConnectionTest	4	עברו
RoomProtocolTest	11	עברו

8.2 תוצאות ההרצה

```
bash gradlew --no-daemon --max-workers=2 test classes startScripts
```

בהרצה שנעשתה לצורך עדכון הספר התקבל BUILD SUCCESSFUL. סך הכול עברו 47 בדיקות, עם 0 כישלונות, 0 שגיאות ו־0 דילוגים. גם classes ו־startScripts הסתיימו בהצלחה. הסביבה הייתה Debian 13.6, OpenJDK 21.0.12.1 ו־Gradle 8.8; הפרויקט מכוון לתאימות Java 17.

8.3 היקף האימות

הבדיקות מכסות אכילה מחייבת ורצף, הכתרה, ניצחון, תחרות בין מצטרפים, סיום משחק, שמירה פעם אחת והתמדה אחרי הפעלה מחדש. צילומי המסך בפרק 16 מופקים מהממשק הנוכחי בעזרת תרחיש הדגמה מקומי. אין בכך בדיקה מלאה של כל אנימציה, סביבת Windows או שני מחשבים פיזיים ברשת.

9. הפעלה וסביבת פיתוח

9.1 סביבת פיתוח

המקורות נמצאים ב־src/main/java, העיצוב ב־src/main/resources/damka.css והבדיקות ב־src/test/java. build.gradle מגדיר Java 17, JavaFX 17.0.6 עם javafx.controls, JUnit 5.10.2 ו־Main. sqlite-jdbc 3.50.3.0 מפעילה את הלקוח, ו־tcpServer היא משימת Gradle להפעלת השרת.

9.2 הרצה ב־VS Code

- פותחים את התיקייה שבה נמצא build.gradle וממתינים לייבוא הפרויקט ותלויותיו.
- פותחים DamkaServer.java ומפעילים Run מעל main. משאירים את השרת פעיל.
- מפעילים Run מעל Main.main פעמיים כדי לפתוח שני חלונות לקוח.
- יוצרים שני חשבונות שונים או נכנסים לחשבונות קיימים. הראשון יוצר חדר והשני מצטרף בעזרת הקוד.

9.3 שלושה מסופים במחשב אחד

```
# Terminal 1
.\gradlew.bat tcpServer

# Terminal 2
.\gradlew.bat run

# Terminal 3
.\gradlew.bat run
```

הפקודות מופעלות מתוך תיקיית הפרויקט. localhost מפנה לאותו מחשב, ולכן די בשרת אחד ובשני חלונות לצורך הדגמה. אם java אינה מזוהה ב־PowerShell, יש להגדיר JAVA_HOME לתיקיית ה־JDK ואת bin ב־Path. אפשר גם להפעיל Main דרך VS Code לאחר שתלויות JavaFX נטענו.

9. הפעלה וסביבת פיתוח — המשך

9.4 שני מחשבים ברשת המקומית

מפעילים את השרת במחשב אחד ומוצאים את כתובת ה־IPv4 שלו באמצעות ipconfig. מאפשרים לתהליך Java לקבל חיבורי TCP בפורט שנבחר, ברשת הפרטית. כל לקוח מופעל עם כתובת מחשב השרת ועם אותו פורט. גם כאשר השרת רץ על מחשב של אחד השחקנים, הלקוח במחשב השני אינו משתמש ב־localhost.

```
.\gradlew.bat run -PserverHost=192.168.1.10 -PserverPort=5000
```

הכתובת היא דוגמה ויש להחליפה בכתובת השרת בפועל. ב־VS Code ניתן להעביר vmArgs של Java: Ddamka.server.host=192.168.1.10- ו־Ddamka.server.port=5000. כדי להפעיל שרת בפורט אחר:

```
.\gradlew.bat tcpServer --args="6000"
.\gradlew.bat run -PserverHost=192.168.1.10 -PserverPort=6000
```

9.5 נתונים, בדיקות ותקלות

קובץ data/damka.db נוצר בתיקיית ההרצה של השרת וממשיך להתקיים אחרי סגירת היישומים. אפשר לשנות את מיקומו באמצעות מאפיין Java בשם damka.database.path. בדיקות מופעלות באמצעות gradlew.bat test.\. ב־Linux או macOS מחליפים את gradlew.bat\. ב־bash gradlew.

JDK של שירות השפה ב־VS Code וה־JAVA_HOME של המסוף הן הגדרות שונות. שירות השפה עשוי לדרוש Java 21, אף שהמשחק מכוון ל־17. במקרה של שגיאת בנייה אין להמשיך להריץ גרסה ישנה. במקרה של חיבור שנכשל בודקים שהשרת פועל, שהכתובת והפורט נכונים ושחומת האש מאפשרת חיבור.

10. סיכום ומקורות

10.1 סיכום

הפרויקט הורחב ממשחק מקומי למערכת הכוללת שני לקוחות, שרת TCP וחשבונות מתמשכים. כללי הדמקה משותפים למשחק הרשת ול־BOT, אך משחק מול אדם אחר מנוהל בשרת. ללקוח תפקיד של בחירה, שליחת בקשה, ציור ומשוב. מנגנון השמירה קושר תוצאה לחשבונות ויוצר ממנה גם היסטוריה וגם סטטיסטיקה.

10.2 תוצאות ומסקנות

הפרדת Game מאפשרת לבדוק חוקיות ללא ממשק. פרוטוקול מבוסס שורות מאפשר לעקוב אחרי בקשות ותשובות, והקשר בין Socket לחשבון מונע מבקשת פרופיל רגילה לבחור משתמש אחר. מזהה ייחודי לכל תוצאה מונע ספירה כפולה. 47 הבדיקות שעברו מספקות אימות של כללים, תחרות בין בקשות, חשבונות ושמירה.

10.3 המשך פיתוח

- להוסיף TLS והגנה מפני ניסיונות כניסה רבים לפני שימוש מחוץ לרשת המקומית.
- להעביר חיפוש BOT לחוט רקע ולבחון שיפור בפונקציית ההערכה.
- להוסיף תמיכה בהתחברות מחדש ובהמשך משחק פעיל, או מנגנון גילוי ניתוק שקט.
- להרחיב בדיקות ממשק ו־LAN, ולהוסיף הדרכה מובנית למשתמש.

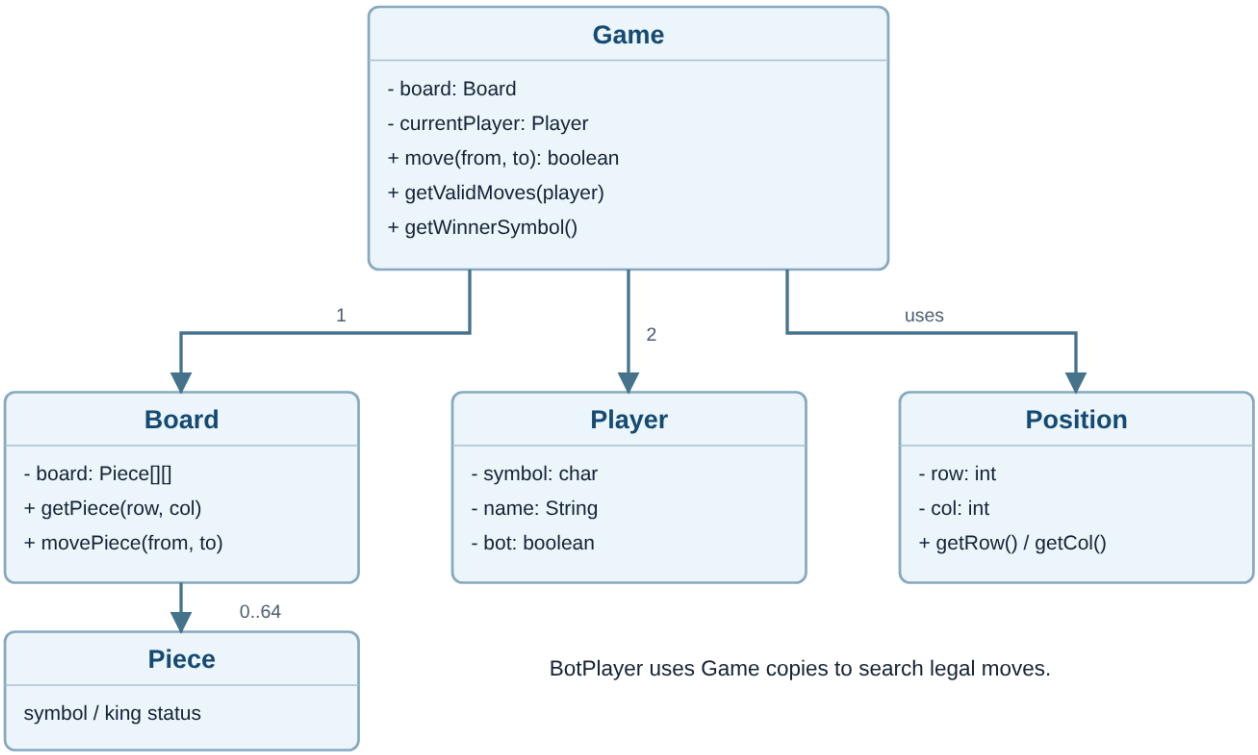
10.4 מקורות

- קוד המקור של MuthannaFrhat/damka-game בענף add-user-profile, קובצי README ו־LEARNING ובדיקות JUnit.
- ספר הפרויקט המצורף Damka_Project_Book_Corrected_v4.pdf והערות המנחה בקובץ Word — בסיס למבנה ולתוכן המעודכן.
- Java SE API: Socket, ServerSocket, Thread, JDBC; תיעוד OpenJFX, SQLite, Gradle ו־JUnit.
- כתובות תיעוד: docs.oracle.com, openjfx.io, sqlite.org, docs.gradle.org, junit.org.

> תיעוד טכני זה מבוסס על המקורות ועל הבדיקות בזמן הכנת הספר. תרחישי ההפעלה המקומיים וצילומי ההדגמה אינם הוכחה לפריסה בענן או לבדיקת רשת בין שני מחשבים פיזיים.

11. תרשים UML וקשרים בין המחלקות

11.1 מודל המשחק והלקוח



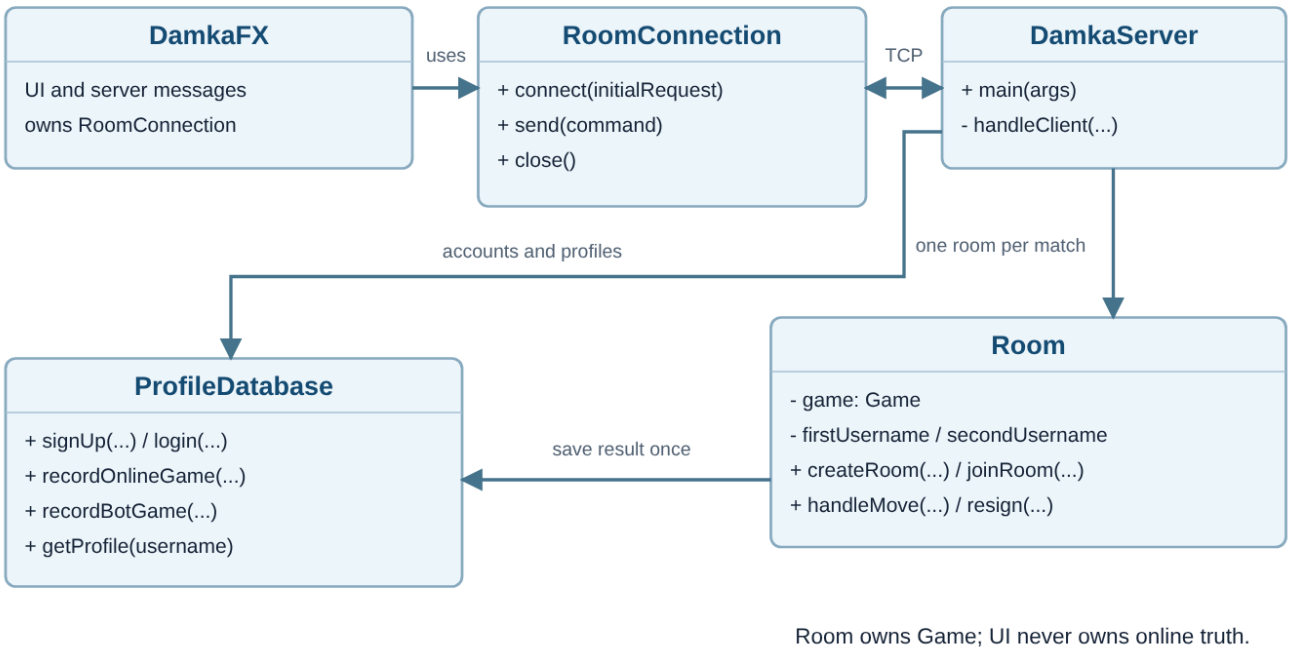
איור 2 — מחלקות המשחק והקשרים העיקריים. Game מחזיקה Board ושני Player; Board מכילה Piece.

Main מפעילה את DamkaFX. DamkaFX משתמשת ב־BoardView להצגת לוח, ב־AccountView לקבלת פרטי חשבון וב־ProfileWindow להצגת פרופיל. במשחק BOT היא מחזיקה Game מקומית ומשתמשת ב־BotPlayer. במשחק מקוון אותו מודל Game נמצא בשרת ולא נוצר לכל מהלך אצל הלקוח.

Position משמשת להעברת מיקום, ו־Game.Move משמשת לתיאור מהלך מאושר. התרשים מציג את הקשרים החשובים להבנת האחריות; אינו מציג את מחלקות הספריות ואת כל רכיבי JavaFX.

11. תרשים UML וקשרים בין המחלקות — המשך

11.2 רשת, חדרים ושמירה



איור 3 — RoomConnection היא תעבורת הלקוח, DamkaServer מנתבת ו־Room מנהלת משחק בשרת.

הקשר בין RoomConnection לבין DamkaServer הוא קשר TCP בין תהליכים, ולא קריאה ישירה למתודה. הלקוח אינו מחזיק Room של השרת. DamkaServer מחזיקה הקשר לכל חיבור, ו־Room משתמשת בערוצי הכתיבה של שני המשתתפים. ProfileDatabase נמצאת בצד השרת ומחזירה ProfileData להעברה.

המפה המשותפת של החדרים מוגנת בנעילת Room.class, ולכל Room נעילה משלה עבור המשחק והמשתתפים. שכבת מסד הנתונים פותחת Connection של JDBC לכל פעולה וסוגרת אותה בסופה.

12. פירוט מעמיק של המחלקות

12.1 Game — סמכות לכללים

move היא שער השינוי במשחק. getValidMoves מייצרת אפשרויות לפי הכיוון והאכילה המחייבת. canSelectPiece עוזרת לממשק BOT לבחור כלי שניתן להזיז.
getCaptureContinuationPosition מחזירה את מיקום הכלי שחייב להמשיך רצף. getLastMove מספקת נתונים לציור התנועה, ו־getWinnerSymbol מחזירה זהות מנצח בלי תלות בטקסט שמוצג במסך.

12.2 Board ו־Piece — נתונים ולא כללים

getPiece מאפשרת לקרוא את מצב התא; isEmpty ו־isInsideBoard תומכות בבדיקות המודל.
movePiece ו־removePiece הן פעולות שינוי פשוטות. כאשר Game קובעת הכתרה היא קוראת Piece.makeKing. גבול אחריות זה מונע מבדיקות התנועה להיות מפוזרות גם בלוח וגם בממשק.

12.3 BotPlayer — החלטה על עותק

findBestMove בוחרת פעולה. minimax מחשבת את ערך תתי־העצים, ו־evaluateBoard מחשבת ניקוד כאשר מגיעים לגבול העומק. ה־BOT אינה מחליטה אילו קפיצות חוקיות; היא מקבלת אותן מ־Game. גם אם חיפוש מצא מהלך, הביצוע בפועל עובר שוב דרך Game.move.

12.4 DamkaFX ו־BoardView — קלט ותצוגה

handleClick מנתבת למצב המקוון או למצב BOT. drawBoard מעבירה תמונת לוח ל־BoardView, ו־applyLocalMove מבצעת מהלך מקומי מאושר. BoardView.showBoard מציירת תמונה, ו־animateMove מציגה את המעבר לפני ציור המצב החדש. קוד זה אינו קובע תור או מנצח בעצמו.

12. פירוט מעמיק של המחלקות — המשך

12.5 DamkaServer ו־Room

serve ממתינה ללקוחות ומפעילה עובד לכל חיבור. handleClient קוראת בקשות, שומרת את זהות החשבון ומנתבת לחדר או למסד. Room.createRoom ו־joinRoom מקצות מקומות; handleMove בודקת שייכות ותור ואז קוראת Game.move. resign, leave ו־playerDisconnected מטפלות בסיום מחוץ לכללי התנועה. broadcastState שולחת לוח ותור.

12.6 RoomConnection

connect מופעלת פעם אחת לכל אובייקט. readMessages מקימה את חיבור ה־Socket וממשיכה לקרוא שורות. send מוסיפה בקשה לתור של עובד כתיבה אחד, וכך שומרת על סדר הפקודות. close סוגרת את התעבורה ועוצרת את עובד הכתיבה. Listener מעבירה אירוע חיבור, הודעה או ניתוק אל DamkaFX.

12.7 AccountView ו־ProfileWindow

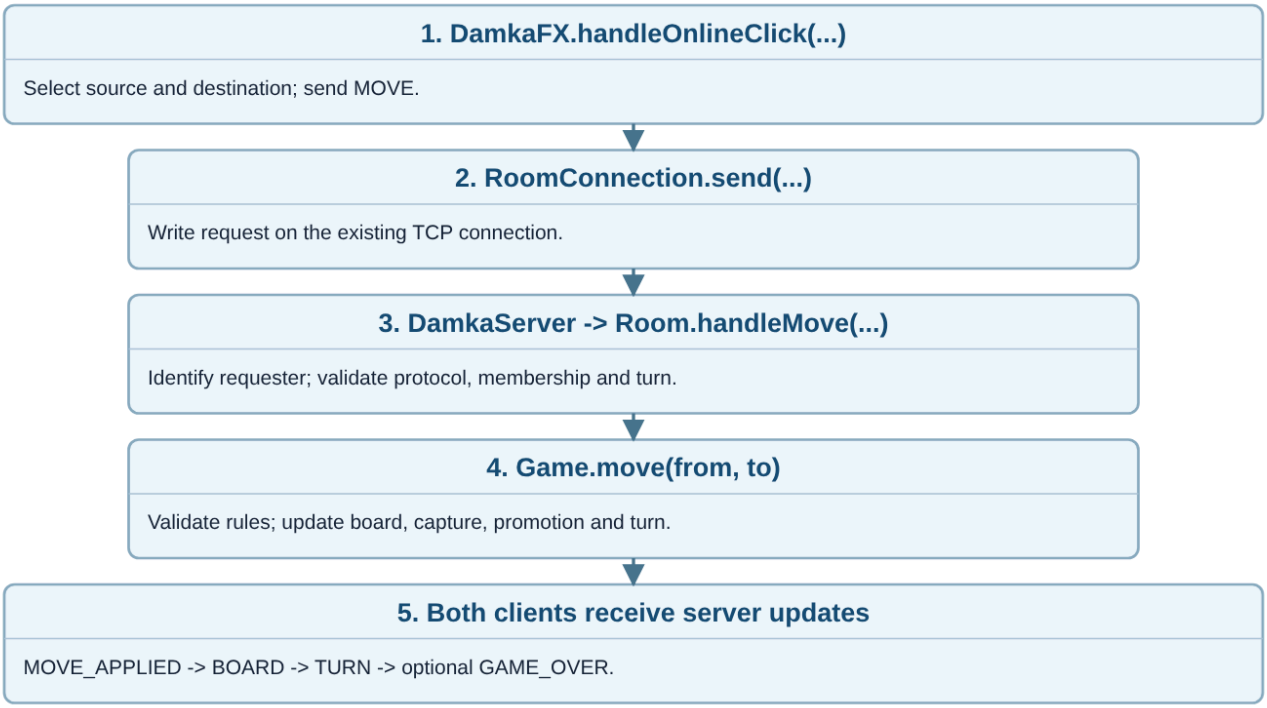
AccountView בודקת קלט מקומי ומחזירה Credentials. DamkaFX.submitAuthentication מנהלת את הבקשה; הטופס עצמו אינו פותח Socket. ProfileWindow.showData מציגה את ProfileData בטבלת היסטוריה ובכרטיסי סטטיסטיקה. פתיחה או Refresh שולחות בקשת פרופיל דרך DamkaFX.

12.8 ProfileDatabase ו־ProfileData

signUp ו־login עוסקות בחשבונות. recordOnlineGame ו־recordBotGame שומרות תוצאות, ו־getProfile מפיקה סטטיסטיקה והיסטוריה. ProfileData.encode ו־decode מסכימות על ייצוג הנתונים כטקסט מקודד. המחלקות הללו מפרידות בין SQL, העברת נתונים והתצוגה שלהם.

13. זרימת פעולה במערכת

13.1 ביצוע מהלך מקוון



איור 4 — מהלחיצה ועד לאותו מצב לוח בשני הלקוחות.

הלקוח שולח מקור ויעד ולא שולח לוח חלופי. DamkaServer מעבירה את הבקשה לחדר של אותו חיבור. Game משנה את הלוח רק כאשר נמצא מהלך חוקי. MOVE_APPLIED מתארת את התנועה, BOARD מוסרת את התמונה החדשה ו־TURN את התור. שני הלקוחות מקבלים את אותו עדכון.

13.2 אנימציה מול סדר הודעות

DamkaFX שומרת הודעות משחק בתור incomingMessages. כאשר BoardView מנפישה MOVE_APPLIED, עיבוד הודעות הלוח הבאות ממתין. בסיום האנימציה מעובדים BOARD ו־TURN. כך אין קפיצה מידית אל מצב הסיום בזמן שהכלי עדיין זז, ושני המצבים BOT ורשת משתמשים באותה תנועה.

13. זרימת פעולה במערכת — המשך

13.3 הרשמה או התחברות

```
AccountView → DamkaFX.submitAuthentication
→ RoomConnection.connect → DamkaServer.authenticate
→ ProfileDatabase.signUp / login
→ AUTH_OK → DamkaFX.handleAccountMessage
```

לאחר AUTH_OK הטופס נסגר ומסך המשחק מוצג. username בהקשר השרת כבר נקבע, והחיבור ממשיך לחדרים ולפרופיל. אחרי AUTH_ERROR מוצגת הודעה, והלקוח יכול לנסות שוב בחיבור חדש.

13.4 יצירה והצטרפות

Create a room שולח CREATE_ROOM. היוצר מקבל קוד ו־PLAYER X. שחקן אחר שולח JOIN_ROOM עם הקוד ומקבל PLAYER O. השרת שולח לשניהם ROOM_READY, אחריה BOARD ראשונית ו־ROOM_JOINED. TURN X מאשרת את המקום של המצטרף; ROOM_READY מודיעה שהחדר מלא והמשחק יכול להתחיל.

13.5 פרופיל ותוצאת BOT

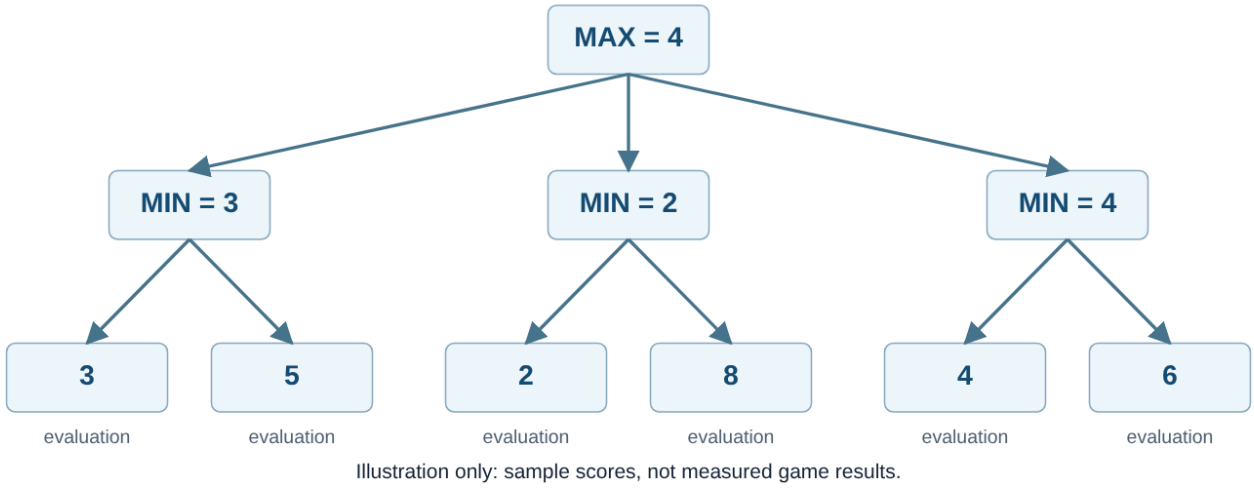
```
Profile / Refresh → PROFILE_REQUEST
→ DamkaServer.sendProfile → ProfileDatabase.getProfile
→ ProfileData.encode → PROFILE → ProfileWindow.showData
```

סיום BOT מפעיל recordBotResult רק לפי תנאי החשבון והחיבור שבהם התחיל המשחק. BOT_RESULT נשלחת ברקע. BOT_SAVED מאשרת שמירה ומאפשרת לרענן פרופיל פתוח. משחק אימון אינו נשלח לאחר כניסה מאוחרת.

14. אלגוריתם Minimax וגיזום Alpha-Beta

14.1 עץ המשחק

כל צומת מייצג מצב Game, וכל קשת היא מהלך חוקי שהוחל על עותק. בצומת שבו תור ה־BOT נבחר הערך הגבוה ביותר, ובצומת של היריב נבחר הנמוך ביותר. הקוד בודק את זהות השחקן הנוכחי בכל מצב; אם אכילה ממשיכה את אותו תור, אין החלפה אוטומטית בין MAX ל־MIN.



איור 5 — דוגמה מספרית: MIN מחזירה 3, 2 ו־4; MAX בוחרת 4. אלו ציוני המחשה, לא תוצאות ניסוי.

14.2 Alpha-Beta

alpha היא הגבול הטוב ביותר שכבר נמצא לצד הממקסם, ו־beta היא הגבול לצד הממזער. בכל בחינה מתעדכן הגבול המתאים. כאשר beta קטנה או שווה ל־alpha, אין צורך להמשיך לבדוק את יתר האפשרויות באותו ענף, כי הן לא ישפרו את הבחירה שכבר זמינה לאחד הצדדים. הגיזום חוסך בדיקות בלי לשנות את עקרון הבחירה של Minimax.

14.3 סיבוכיות וזיהוי סיום

כאשר יש בממוצע b אפשרויות בכל מצב ועומק d, מספר המצבים בחיפוש מלא הוא מסדר גודל O(b^d). גיזום יכול להפחית עבודה, בהתאם לסדר המהלכים ולמצבים. ניצחון BOT מקבל 10000 והפסד מקבל -10000. אם אין סיום והעומק נגמר, evaluateBoard מחזירה הערכת חומר.

15. רמות קושי ופונקציית הערכה

15.1 אותו אלגוריתם בשלוש רמות

רמה	עומק	משמעות
Easy	1	בחינת תוצאת המהלך המועמד בעומק הקטן ביותר.
Medium	2	בחינת שכבה נוספת של מהלכים חוקיים.
Hard	3	בחינת שלוש שכבות, בעלות חישוב גבוהה יותר.

העומק פוחת לאחר כל מהלך או קפיצה. כאשר יש רצף אכילות, כמה שכבות עשויות להשתייך לאותו שחקן; לכן עומק 3 אינו הבטחה לשלושה תורות מלאים לסירוגין. הרמות אינן משתמשות באלגוריתמים שונים ואין למידת מכונה.

15.2 הערכת חומר

```
score = 3 × (BOT men − opponent men)
      + 6 × (BOT kings − opponent kings)
```

כלי רגיל שווה 3 ומלך שווה 6. לדוגמה, BOT עם חמישה כלים ומלך מול ארבעה כלים ומלך מקבלת ציון 3. ערך המלך גבוה כי הוא יכול לנוע ולאכול לשני הכיוונים. יחס 2:1 הוא בחירת תכנון פשוטה; לא תועד ניסוי השוואה שמוכיח שהוא אופטימלי.

15.3 גבולות ההערכה

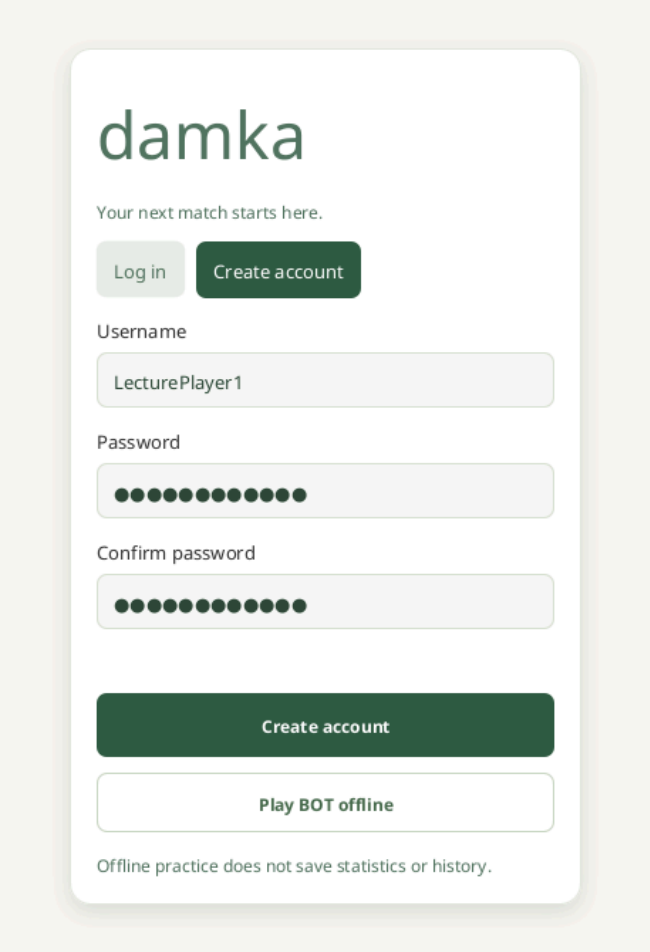
הניקוד אינו מעניק ערך נפרד למרכז, ליניידות, להגנה, למרחק מהכתרה או לאיום עתידי. שני לוחות שונים עם אותו הרכב כלים יכולים לקבל אותו ציון. אפשר לשפר את ההערכה בעתיד, אך לצורך הפרויקט הפונקציה קלה להבנה ולבדיקה.

15.4 העתקת מצב

כל ענף מתחיל ב־new Game(game), הכוללת את חובת המשך האכילה. לכן אפשר לבדוק אפשרות, לעבור לאחרת ולבסוף לבצע רק את המהלך שנבחר על המשחק המקומי האמיתי.

16. מסכי המערכת

16.1 מסך החשבון



איור 6 — הרשמה בממשק הנוכחי: שם משתמש, סיסמה ואישור סיסמה.

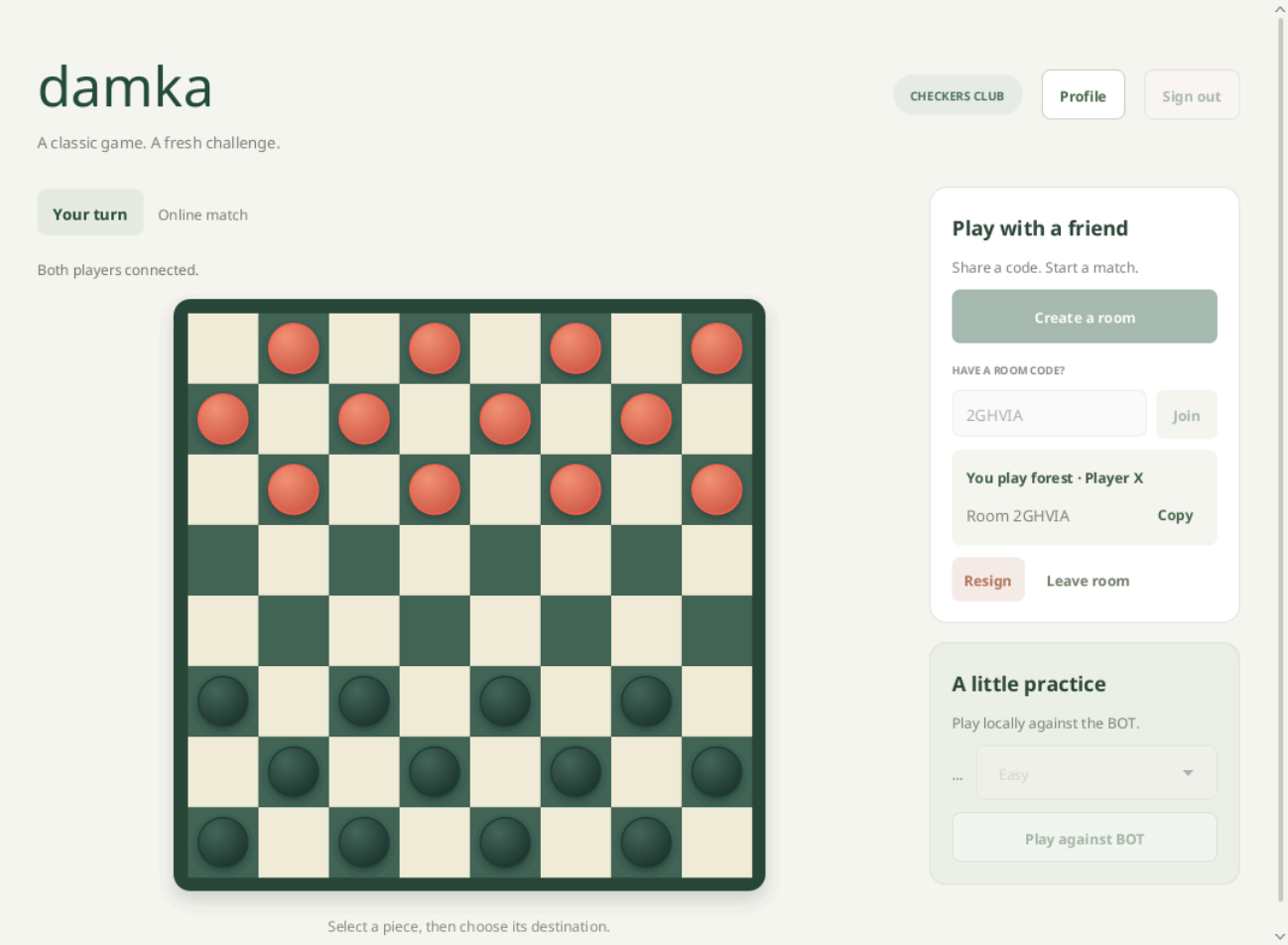
אפשר לעבור בין Log in לבין Create account. הטופס מציג שגיאת קלט או הזדהות במקום ומושבת במהלך הבקשה. Play BOT offline מאפשר להתחיל אימון ללא שרת או חשבון. בצילומים נעשה שימוש בחשבונות הדגמה ובמסד זמני.

16.2 אחרי התחברות

המשתמש עובר למסך המשחק. לפני ששני שחקנים הצטרפו, אין משחק רשת פעיל והלוח ממתין. פעולות Create a room ו־Join משתמשות בחיבור המאומת שכבר נפתח במהלך ההזדהות.

16. מסכי המערכת — המשך

16.3 משחק רשת



איור 7 — חדר עם שני שחקנים: לוח השרת, התור, קוד החדר וכפתורי סיום.

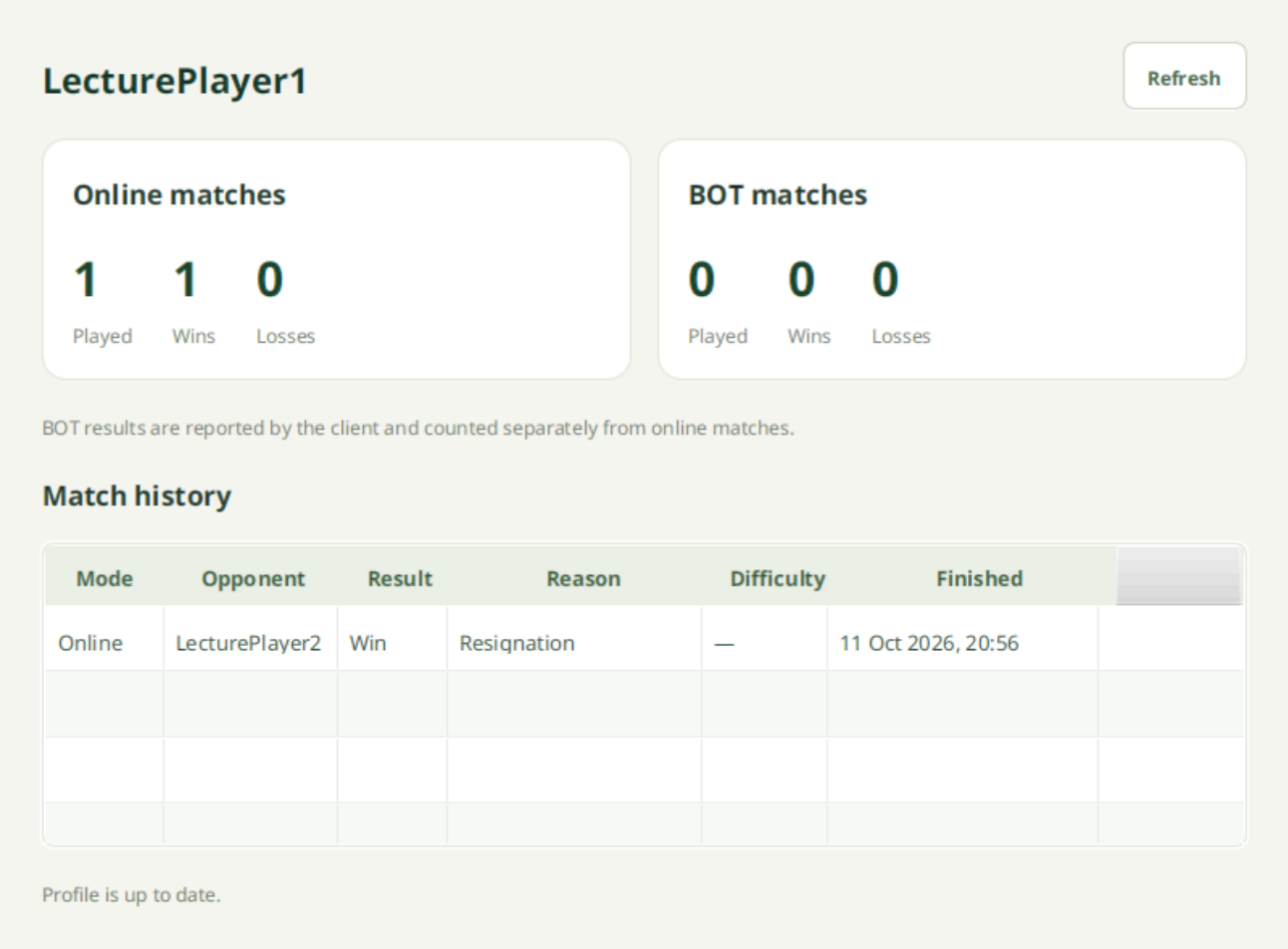
הצילום הופק אחרי הרשמה של לקוח JavaFX והצטרפות לקוח פרוטוקול נוסף לאותו שרת מקומי. התקבלו PLAYER X/O, ROOM_READY, BOARD ו־TURN. ניתן להעתיק את הקוד באמצעות Copy. אין במסך שדות localhost ופורט; הם מוגדרים בהפעלה.

16.4 מעבר בין מצבים

משחק מול BOT מופעל באמצעות Play against BOT לאחר יציאה מחדר. Create או Join מחזירות למצב מקוון כאשר המשתמש מחובר. הממשק מונע שינוי מצב באמצע בקשת חדר או מהלך שעדיין מתבצעים.

16. מסכי המערכת — המשך

16.5 פרופיל ונתוני משחק

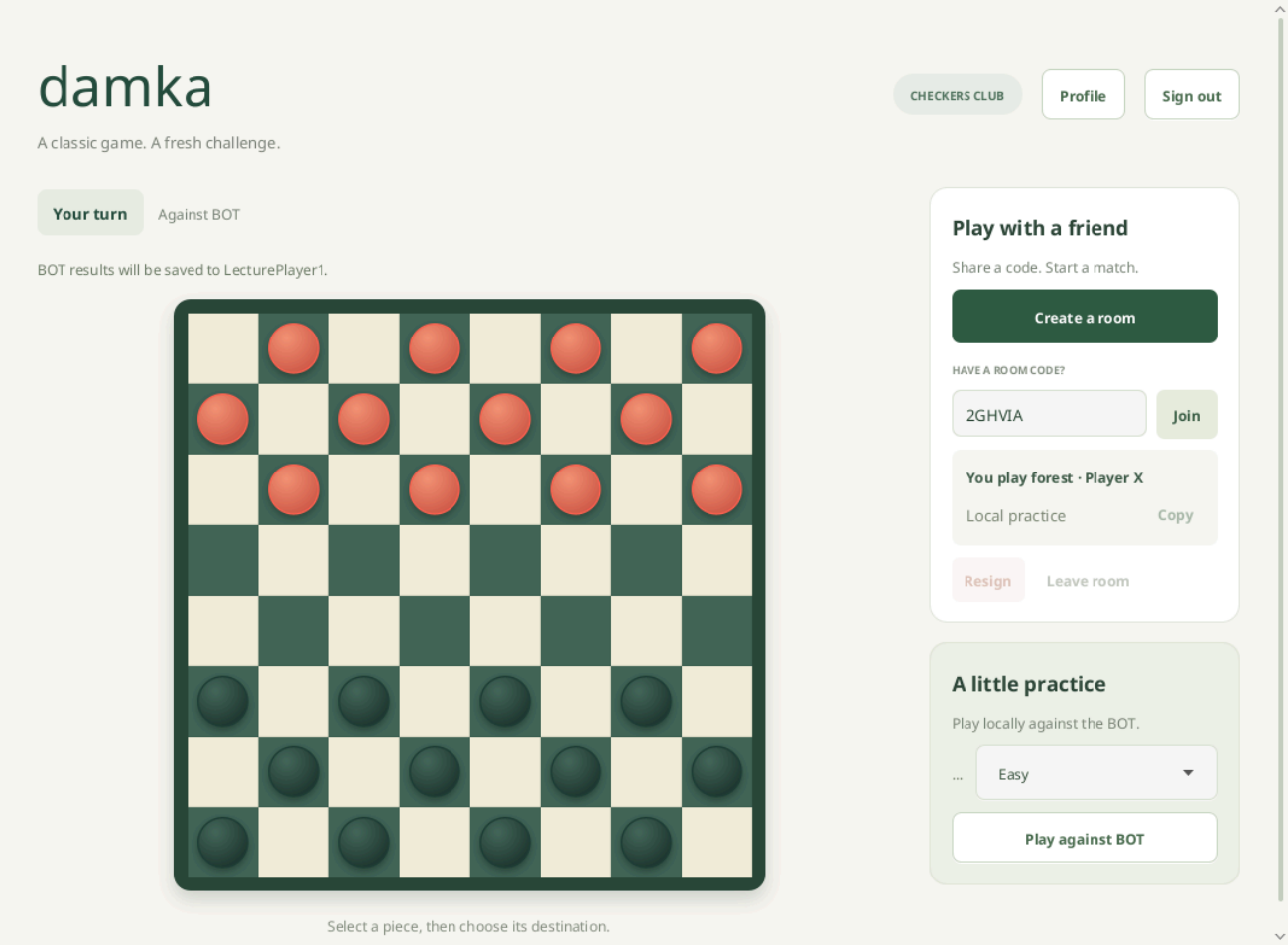


איור 8 — חלון Profile לאחר משחק הדגמה שהסתיים בפרישת היריב.

החלון מציג Online matches ו־BOT matches בנפרד, ובהם Played, Wins ו־Losses. מתחת מופיעה היסטוריה עם יריב, תוצאה, סיבת סיום, רמת קושי וזמן. Refresh מבקשת מידע חדש מהשרת. כדי לחזור למשחק סוגרים את חלון הפרופיל או עוברים לחלון המשחק שנותר פתוח.

16. מסכי המערכת — המשך

16.6 תרגול מול BOT



איור 9 — משחק BOT בממשק הנוכחי לאחר התחברות; רמות הקושי נמצאות בכרטיס הצד.

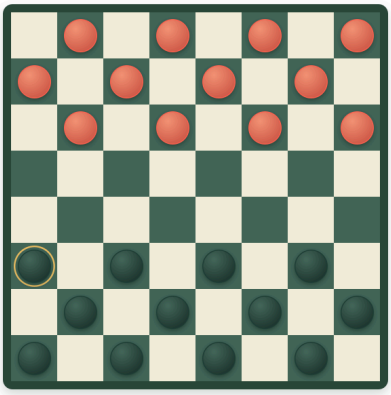
במצב BOT הלוח מנוהל מקומית ומוצגות רמות Easy, Medium ו־Hard. כשהמשחק התחיל בחשבון פעיל, מוצגת הודעה שהתוצאה תישמר עבורו. ללא חשבון או אחרי ניתוק מוצג שזהו Practice ללא שמירת סטטיסטיקה.

16.7 משחק ללא חשבון

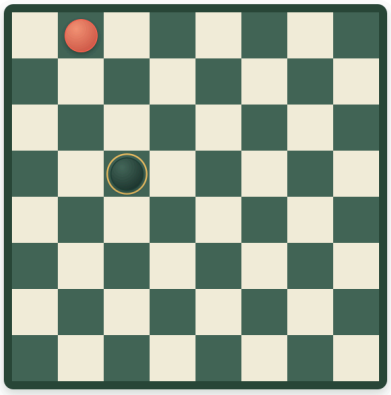
Play BOT offline במסך הפתיחה מובילה ישירות למשחק תרגול. הכפתורים המקוונים והפרופיל אינם זמינים ללא חשבון. אפשר לפתוח Log in מתוך מסך המשחק, אך ההתחברות אינה שומרת בדיעבד את התוצאה של משחק שהתחיל כאימון. להפיכת משחק BOT למשחק מתועד מתחילים משחק חדש אחרי כניסה.

17. מצבי משחק חזותיים

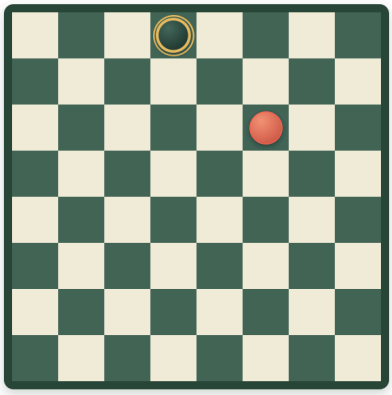
17.1 בחירה, אכילה והכתרה



איור 10 — טבעת בחירה



איור 11 — אחרי אכילה



איור 12 — מסגרת מלך

האיורים מציגים BoardView עם מצבי הדגמה שנוצרו במודל המשחק: בחירה של כלי, תוצאת אכילה ומלך. הם מדגימים את הקשר בין הסימנים בתמונת הלוח לבין המראה, ואינם צילומי משחק של משתמשים אמיתיים. selectedPosition מוסיפה טבעת מסביב לכלי; אות גדולה מוסיפה מסגרת מלך.

17.2 שגיאה ותיקון

מהלך שגוי ברשת מקבל ERROR ואינו משנה את תמונת הלוח. כרטיס השגיאה צף מעל התוכן, כך שהצגתו אינה משנה את גודל הלוח. המשתמש יכול לסגור אותו ב־×. לאחר פעולה מוצלחת מוצגת הודעת מצב והטעות הקודמת אינה נשארת לצד מהלך תקין.

17.3 אנימציה משותפת

הכלי הנייח במקור מוסתר, ובשכבה שמעל הלוח מופיע כלי נע באותו עיצוב. ההתקדמות מחושבת מ־0 עד 1 בין מרכזי התאים. אם נאכל כלי, הוא נעלם בהדרגה. בסוף התנועה מוצג המצב המאושר החדש. זהו אותו רכיב ואותו מסלול ציור בשני מצבי המשחק.

17.4 סיום ברור

ניצחון, פרישה, עזיבה או ניתוק מוצגים עם סיבה. כפתורי מהלך ופרישה מותאמים למצב החדר. אחרי סיום אפשר ליצור או להצטרף לחדר אחר בחיבור החשבון הקיים. סיום חדר אינו זהה בהכרח לסגירת Socket.

18. אתגרים ותהליך העבודה

18.1 הרחבה מעבר למחשב אחד

במשחק המקומי שינוי Game וציור הלוח התרחשו באותו תהליך. המעבר ל־TCP דרש להחליט מי קובע את מצב המשחק, כיצד כל לקוח מזהה את תורו וכיצד מעבירים עדכון לשניהם. הפתרון הוא Game אחת לכל Room בשרת, בקשות MOVE מהלקוחות ותמונת מצב משותפת שנשלחת אחרי אישור.

18.2 כללים ורצף אכילות

אתגר מרכזי היה לשמור על אותה חוקיות בממשק, בשרת ובחיפוש BOT. getValidMoves מרכזת את יצירת האפשרויות, ו־captureContinuationPosition מגבילה רצף לאותו כלי. בנאי העתקה מלא שומר את החובה בחיפוש. הבדיקות כוללות דחיית מעבר לכלי אחר והכתרה באמצע רצף.

18.3 חוטי ביצוע וממשק

readLine יכולה להמתין זמן רב, ולכן הועברה לחוט קורא. כתיבה מועברת לעובד יחיד לשמירת סדר. Platform.runLater מעבירה טיפול במסרים אל חוט JavaFX. במקביל, תור הודעות המשחק מונע החלפת לוח באמצע אנימציה. פעולת close משחררת קריאה ממתינה ומסיימת את התעבורה.

18.4 שמירה וזיהוי כפילויות

נדרש להבחין בין תוצאה שנקבעה בשרת לבין BOT המדווחת מהלקוח, ובין משחק בחשבון לבין אימון ללא חשבון. מזהה UUID, שורה אחת לכל תוצאה וסטטיסטיקה הנגזרת ממנה מונעים מונים סותרים. אחרי שגיאת שמירה נמסרת הודעה במקום להציג כאילו התוצאה נרשמה.

18.5 AI והתרומה האישית

הנוסח המתאר את השימוש ב־AI ואת חלוקת העבודה בין שני המגישים יושלם עמם בפרק 21. אין כאן ייחוס משימות או טענה על היקף כתיבה עצמאי שלא אושרו.

19. תקשורת TCP וניהול חיבורים

19.1 TCP, ServerSocket ו־Socket

TCP מעבירה רצף בתים מסודר בין שני קצוות. ServerSocket היא נקודת ההאזנה בשרת: new ServerSocket(port) פותחת פורט ומאפשרת להמתין לחיבורים. accept מחכה ללקוח ומחזירה Socket נפרדת של החיבור שנוצר. האזנה אינה החיבור לשחקן מסוים; היא מאפשרת לקבל עוד חיבורים.

בצד הלקוח RoomConnection יוצרת Socket ללא חיבור. readMessages קוראת socket.connect עם InetSocketAddress, וכאן מתבצע ניסיון ההתחברות בפועל. אחרי הצלחה שני הצדדים יכולים לקרוא ולכתוב דרך ערוצי ה־Socket. השרת ממשיך ל־accept הבא בזמן שעובד מטפל בשחקן שהתקבל.

19.2 כתובת ופורט

כתובת IP מזהה את מחשב השרת, ופורט מזהה את נקודת ההאזנה של היישום באותו מחשב. localhost מתייחס למחשב שבו התוכנית רצה. כתובת LAN כמו 192.168.1.10 מאפשרת להגיע למחשב אחר ברשת המקומית. הלקוח חייב להשתמש בכתובת ובפורט של השרת, ולא בכתובת של היריב.

19.3 מתי נפתח החיבור

בגרסה הסופית החיבור נפתח בהרשמה או בהתחברות, וממשיך לשמש חדרים, מהלכים, פרופיל ותוצאות. BOT אין פתיחת Socket לכל לחיצה. הפרמטר 5000 בקריאת connect הוא זמן מרבי להקמת החיבור, במילישניות. אין כאן setSoTimeout לקריאה, ולכן שחקן אינו מנותק בגלל שבילה יותר מחמש שניות בהחלטה.

> TCP מספקת סדר בתים; היא אינה מחלקת אוטומטית את הזרם להודעות משחק ואינה מצפינה אותו. גבולות ההודעות נקבעים בפרוטוקול השורות.

19. תקשורת TCP וניהול חיבורים — המשך

19.4 מקריאת בתים להודעה

getInputStream מחזירה זרם בתים שהגיעו מהחיבור. InputStreamReader ממירה אותם לתווים בקידוד UTF-8. BufferedReader מאפשרת readLine: היא מחזירה שורה כשהגיע סוף שורה, או את התווים שנותרו בסיום הזרם. null מוחזר רק בסיום הזרם בלי תווים שנותרו; כשל קריאה עלול לגרום ל־IOException. בצד השני OutputStreamWriter ממירה תווים לבתים, ו־PrintWriter.println מוסיפה שורה חדשה ושולחת הודעה.

```
BufferedReader input = new BufferedReader(
    new InputStreamReader(socket.getInputStream(), UTF_8));
PrintWriter output = new PrintWriter(
    new OutputStreamWriter(socket.getOutputStream(), UTF_8), true);
```

הקוד למעלה הוא תמצית של עטיפות הזרמים; בקוד הפרויקט הקבוע מופיע כ־StandardCharsets.UTF_8. האובייקט החיצוני נותן פעולה נוחה על האובייקט הפנימי. true מפעילה flush אוטומטי ב־println, כך שהשורה לא נשארת רק במאגר מקומי.

19.5 קידוד הלוח

Room.encodeBoard עוברת על 64 התאים לפי שורות. תא ריק הופך לנקודה, וכלי הופך לסימנו. BOARD נשלחת עם המחרוזת. הלקוח מוצא את התא באמצעות row × 8 + col. אין כאן יצירת Board משותפת בזיכרון: שני המחשבים מקבלים תיאור טקסט של אותו מצב.

סימן	משמעות
.	משבצת ריקה
x / o	כלי רגיל השייך ל־X / O
X / O	מלך השייך ל־X / O

זהו snapshot לציור. שינוי שלו אצל הלקוח לא יהיה מהלך מוכר בשרת. מהלך חוקי מתחיל בפקודת MOVE ומתקבל רק אחרי אימות Game. פרופיל, לעומת זאת, נארז כטקסט מובנה ומקודד Base64 כדי להעבירו בשורה אחת.

19. תקשורת TCP וניהול חיבורים — המשך

19.6 פרוטוקול החשבון והמשחק

בקשה או אירוע	תשובה	תפקיד
SIGNUP <username> <passwordBase64>	AUTH_OK <username> / AUTH_ERROR <detail>	יצירת חשבון ואימות החיבור; הסיסמה מועברת בקידוד Base64, שאינו הצפנה.
LOGIN <username> <passwordBase64>	AUTH_OK <username> / AUTH_ERROR <detail>	כניסה לחשבון קיים; זהות החשבון נשמרת בצד השרת עבור אותו Socket.
CREATE_ROOM	ROOM_CREATED <code>, PLAYER X	יצירת חדר בן שני מקומות והקצאת היוצר לשחקן X.
JOIN_ROOM <code>	ROOM_JOINED <code>, PLAYER O	אישור הצטרפות וקביעת השחקן המצטרף כ־O; שגיאה נשלחת אם הקוד שגוי, החדר מלא או החשבון כבר תופס מקום.
לא פקודה; אירוע הצטרפות	ROOM_READY <code>, BOARD <64 symbols>, TURN X	שידור לשני השחקנים לאחר הצלחת ההצטרפות; מציין שהמשחק מתחיל ממצב השרת.
MOVE <fromRow> <fromCol> <toRow> <toCol>	MOVE_APPLIED <fromRow> <fromCol> <toRow> <toCol> <capturedRow> <capturedCol>, BOARD <snapshot>, TURN <X|O>	המהלך המאושר נשלח לשני הלקוחות. באין אכילה capturedRow ו־capturedCol הם -1.
MOVE לא תקין	ERROR <detail>	תשובה למבקש בלבד; מצב המשחק אינו משתנה ולא משודר לוח חדש.
מהלך שמסיים משחק	GAME_OVER <X|O> RULES	שני הלקוחות מקבלים מנצח לאחר עדכון הלוח. כשל שמירה מדווח ב־RESULT_ERROR.

הסימנים שבסוגריים זוויתיים מתארים שדות, ואינם נשלחים כפשוטם. כל תשובה ברשימה היא שורה נפרדת. למשל אחרי MOVE מאושרת נשלחות MOVE_APPLIED, אחריה BOARD ואחריה TURN. כאשר אין כלי שנאכל, קואורדינטות האכילה הן -1,-1.

שמות המשתמשים וסמל השחקן נקבעים בשרת מתוך החיבור. MOVE אינה מכילה שם משתמש או סמל שניתן לבחור באופן חופשי. ERROR אינה מסיימת אוטומטית את החיבור, ולכן אפשר לבצע בקשה תקינה לאחר דחייה.

19. תקשורת TCP וניהול חיבורים — המשך

19.7 סיום, פרופיל ותוצאות

בקשה או אירוע	תשובה	תפקיד
RESIGN	GAME_OVER <X|O> RESIGNATION	פרישה ממשחק פעיל מעניקה ניצחון ליריב.
LEAVE_ROOM	ROOM_LEFT ליריב; GAME_OVER <X|O> OPPONENT_LEFT אם המשחק התחיל	היוצא מקבל אישור. בחדר המתנה אין תוצאת משחק.
לא פקודה; סיום חיבור	GAME_OVER <X|O> OPPONENT_DISCONNECTED	יריבו של שחקן שנותק מנצח במשחק שהתחיל וטרם נסגר.
PROFILE_REQUEST	PROFILE <encodedProfile> / PROFILE_ERROR <detail>	שליפת נתוני החשבון המחובר, הסטטיסטיקות וההיסטוריה; payload מקודד Base64.
BOT_RESULT <uuid> <WIN|LOSS> <difficulty> <finishedAt>	BOT_SAVED <uuid> / BOT_ERROR <uuid> <detail>	שמירת תוצאה מקומית תחת החשבון המאומת; difficulty בטווח 1 עד 3, הזמן בפורמט Instant.
לא פקודה; כשל שמירת תוצאה מקוונת	RESULT_ERROR <detail>	מבהיר שהמשחק הסתיים אבל תוצאתו לא נשמרה.

19.8 Thread וסנכרון

Thread הוא חוט ביצוע שיכול להמתין לרשת בזמן שחוט אחר ממשיך לעבוד. השרת מפעיל אחד לכל לקוח. synchronized (Room.class) מגן על מפת החדרים המשותפת; synchronized (room) מגן על חדר מסוים. כך שני מצטרפים אינם תופסים את אותו מקום ושני מהלכים אינם משנים יחד את אותו תור. הסרה מהמפה מתבצעת לאחר שחרור נעילת החדר כדי לשמור סדר נעילות.

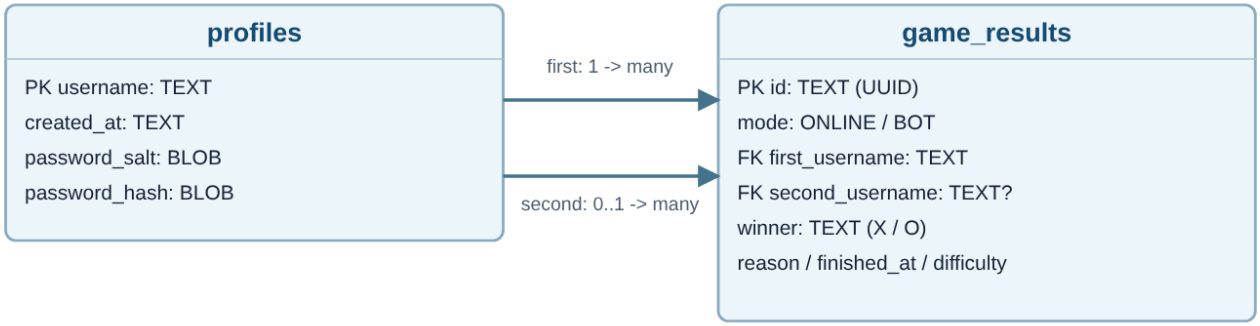
19.9 מחזור חיים ושגיאות

try (Socket connection = client) נותנת שם מקומי לאותו חיבור ומבטיחה close ביציאה מהבלוק, גם בחריגה. בלקוח close סוגרת Socket ומשחררת readLine ממתינה. worker של השרת הוא חוט רגיל; חוטי RoomConnection הם daemon כדי שלא ישאירו את תהליך הלקוח חי לבד. אין heartbeat או שחזור חדר אחרי הפסקת השרת. ניתוק שקט עשוי להתגלות רק כשהמערכת או פעולת רשת מזהות כשל.

20. חשבונות, SQLite ופרופיל משתמש

20.1 מודל השמירה

SQLite שומרת את המידע בקובץ data/damka.db במחשב השרת. JDBC מחברת בין Java למסד. בבנאי ProfileDatabase נוצרות התיקייה והטבלאות אם הן אינן קיימות. אין טבלת משחקים פעילים ואין שמירת מצב חדר לשחזור; נשמרים חשבונות ותוצאות שהסתיימו.



איור 13 — כל תוצאה שייכת לחשבון ראשון; במשחק ONLINE גם לחשבון שני.

טבלה	שדות עיקריים
profiles	username (PK), created_at, password_salt, password_hash
game_results	id (PK), mode, first_username (FK), second_username (FK), winner, reason, finished_at, difficulty

mode היא ONLINE או BOT. במשחק ONLINE שני חשבונות נדרשים ו־difficulty היא 0. ב־BOT המשתמש הוא הראשון, second_username היא NULL והקושי 1 עד 3. winner הוא X או O. סיבת הסיום נשמרת בנפרד, כדי להבחין בין ניצחון לפי כללים לבין פרישה או ניתוק.

20. חשבונות, SQLite ופרופיל משתמש — המשך

20.2 חיבורים ו־SQL

כל פעולה פותחת Connection של JDBC וסוגרת אותה באמצעות try-with-resources. WAL מאפשר לקוראים לעבוד לצד כתיבה, busy_timeout מאפשר המתנה קצרה לנעילת מסד, ו־foreign_keys מפעילה בדיקת קשרים. PreparedStatement מפרידה בין SQL לבין פרמטרי המשתמש: סימני ? מקבלים ערכים דרך setString או setBytes.

> במסד הנוכחי אין בדיקות שדרוג למסד ישן שבו חסרים שדות סיסמה. הטבלאות נוצרות לפי הסכמה של גרסת החשבונות.

20.3 הרשמה

הטופס מקבל שם בן 1 עד 24 אותיות, ספרות, קו תחתון או מקף וסיסמה באורך 8 עד 128 תווים. אותיות Unicode מותרות. שמות תלויים בגודל אות, ולכן Alice ו־alice הם חשבונות נפרדים. הטופס בודק אישור סיסמה; השרת בודק שוב מבנה, קידוד ואורך לפני פעולת SQL.

signUp יוצרת salt אקראי בן 16 בתים ומחשבת PBKDF2WithHmacSHA256 עם 600,000 איטרציות ופלט 256 סיביות, כלומר 32 בתים. היא שומרת את השם, ה־salt וה־hash. INSERT עם ON CONFLICT(username) DO NOTHING אינו דורס חשבון קיים. executeUpdate מחזירה 1 רק כאשר נוצרה שורה, ואז נשלחת AUTH_OK והלקוח גם נכנס לחשבון.

20.4 התחברות

login שולפת password_salt ו־password_hash של השם באמצעות SELECT. rows.getBytes(1) קוראת את העמודה הראשונה בתוצאת השאילתה — ה־salt — ו־getBytes(2) את ה־hash; אלו אינדקסים של תוצאת SELECT, לא מספר שורה בטבלה. מחושב hash מהסיסמה שנמסרה ומה־salt השמור, ומשווים אותו באמצעות MessageDigest.isEqual.

20.5 זהות החיבור

אחרי אימות מוצלח DamkaServer שומרת username בהקשר של ה־Socket. כל יצירת חדר, בקשת פרופיל או תוצאת BOT משתמשת בזהות הזו. לא ניתן לשנות משתמש באותו חיבור מאומת. Sign out סוגרת את החיבור, והתחברות חדשה יוצרת חיבור חדש. אותו חשבון אינו רשאי לתפוס את שני המושבים בחדר.

> הסיסמה אינה נשמרת כטקסט במסד, אך חיבור ה־TCP עצמו אינו מוצפן. Base64 היא קידוד הפיך, ולא הגנה על הסיסמה בדרך. TLS והגבלת ניסיונות כניסה נדרשות להרחבה מחוץ להיקף ה־LAN הנוכחי.

20. חשבונות, SQLite ופרופיל משתמש — המשך

20.6 רישום משחק פעם אחת

עם הצטרפות שחקן שני Room יוצרת matchId מסוג UUID. סיום לפי הכללים, פרישה, עזיבה או ניתוק מובילים ל־recordOnlineGame. נכתבת שורה אחת עם שני המשתתפים, מנצח, סיבה וזמן. חדר שנסגר לפני שהגיע שחקן שני אינו יוצר תוצאה. אם אותה תוצאה נשלחת שוב באותו id, היא אינה נספרת מחדש; ניסיון להשתמש באותו id לפרטים שונים נדחה.

20.7 סטטיסטיקה והיסטוריה

getProfile קוראת את כל התוצאות שבהן השם הוא המשתתף הראשון או השני. לפי המושב וסמל winner נקבעת Win או Loss. סכומי הניצחונות וההפסדים מחושבים בנפרד עבור ONLINE ו־BOT, והם כוללים את כל התוצאות. ההיסטוריה ממוינת מהחדש לישן ומוגבלת ל־100 הרשומות האחרונות. כל הנתונים מחושבים מתוך אותה קריאה, בלי טבלת מונים נפרדת.

PROFILE_REQUEST אינה כוללת שם משתמש. sendProfile בשרת משתמשת בחשבון המאומת, יוצרת ProfileData ושולחת PROFILE עם הלקוח מפענח encode. ב־handleAccountMessage ומעביר ל־ProfileWindow.showData. הפלט נכתב ל־Socket של הלקוח; הוא אינו הודעה שאמורה להופיע במסוף השרת.

20.8 תוצאות BOT והיעדר שרת

startBotGame שומרת את שם החשבון, אובייקט החיבור והקושי בתחילת המשחק.
recordBotResult שולחת רק אם אותו חשבון ואותו חיבור עדיין פעילים. השרת בודק את מבנה התוצאה ושומר אותה תחת זהות החיבור, אבל אינו מריץ מחדש את משחק BOT כדי להוכיח את הניצחון. לכן התוצאות מוצגות בנפרד מ־PvP שמאומת בשרת.

כאשר אין שרת או אין התחברות, משחק BOT הוא אימון בלבד: אין היסטוריה, סטטיסטיקה, קובץ תוצאות ממתינות או שליחה אחרי כניסה מאוחרת. לאחר ניתוק אפשר להמשיך תרגול מקומי, אך תוצאה זו אינה נרשמת בפרופיל.

21. שימוש בכלי AI והתרומה האישית

> פרק זה מיועד להשלמה עם שני המגישים לפני הגשת הספר. התוכן נדחה במפורש בתהליך העבודה, ולכן אין לייחס כאן משימות או תרומה אישית שלא אושרו.

21.1 דרישת המנחה

בהתאם להנחיית המנחה, הספר צריך לכלול תיאור של היקף השימוש ב־AI, אופן השימוש בכלים ומה הייתה התרומה האישית בפרויקט. התיאור צריך להתייחס לפיתוח עצמו ולא להסתפק באמירה כללית שנעשה שימוש בעזרה.

21.2 פרטים להשלמה

- באילו כלי AI נעשה שימוש, ובאילו שלבים של הפרויקט.
- אילו פעולות נעזרו ב־AI: למידה, תכנון, כתיבה, תיקון, בדיקה או תיעוד — לפי מה שנעשה בפועל.
- כיצד נבדקו ההצעות ומה שונה או נדחה במהלך העבודה.
- מה הייתה תרומתו בפועל של כל אחד משני המגישים בתכנון, בפיתוח, בבדיקות ובהבנת הקוד.
- דוגמה אמיתית אחת לתהליך שבו הצעת AI נבחנה ונערך בה שינוי.

21.3 מצב הפרק

טרם נוסח. אין בספר חלוקת עבודה מומצאת, אחוזי שימוש לא מבוססים או טענה שמישהו כתב רכיב מסוים בלי אימות. יתר פרקי הספר מתארים את המערכת הטכנית הנוכחית וניתנים לקריאה ולבדיקה בפני עצמם.